

Online Messaging Authorship Analysis with Doc2Vec and TFIDF

Alex Oshima
UCSD ECE Undergrad
aoshima@ucsd.edu

Akhil Pillai
UCSD ECE Undergrad
avpillai@ucsd.edu

ABSTRACT

‘lol,’ ‘LOL,’ ‘Lol,’ or opting for ‘haha’—the internet and the screens that separate us strip away many social cues that humans have evolved to pick up on. Consequently, individuals try to convey their messages through alternative means, whether it's through capitalization, emojis, or even strings like ‘jkflasjdfl.’ This paper aims to explore these tendencies, examining just how unique our texting styles truly are. The objective is to investigate whether these differences can be leveraged to classify the author of a particular message. Using Doc2Vec, we strive to capture both the potential syntax and meaning of messages. Additionally, TF-IDF is employed to identify the rarity of certain words, helping us pinpoint unique vocabularies of individuals. Furthermore, we gather additional heuristic features from the messages. Finally, an SVM is utilized to combine all these features and classify the messages.

CCS Concepts

• Computing methodologies→Machine Learning→Learning paradigms→Supervised learning

Keywords

Term Frequency-Inverse Document Frequency; Doc2Vec; Word2Vec; Logistic regression; Support Vector Machine

1. INTRODUCTION

Author identification, the process of determining the author of a given body of text, holds numerous real-life applications. It can assign a name to a work when the author is unknown, offering insights into historical figures lacking proper documentation. Additionally, it aids in detecting plagiarism to some extent and helps individuals gain a deeper

understanding of their writing styles by providing quantifiable feedback.

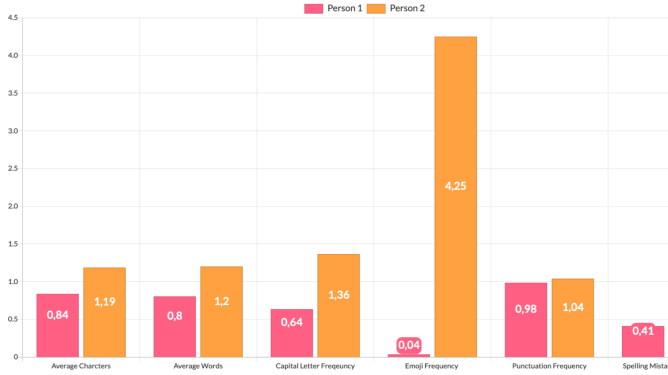
In this study, we trained a support vector machine with a radial basis function kernel using a labeled dataset comprising messages from two distinct individuals, hereafter referred to as Person 1 and Person 2. For each message, we created a feature set encompassing message heuristics, the maximum term frequency-inverse document frequency (TF-IDF) value of each word for each person, and the output of a Doc2Vec model on the data. We opted for a support vector machine due to its effective use of support vectors, streamlining the classification process. This ensemble implementation yielded results more accurate than some popular deep learning approaches, suggesting that a sole deep learning approach might not be the most effective strategy for this task.

2. DATA ANALYSIS

We gathered 25,867 messages from Person 1 and 25,067 messages from Person 2 for model training, extracted from the Discord online chat platform. To enhance our understanding of the dataset, we conducted exploratory analysis, revealing intriguing patterns. Notably, Person 2 incorporates a significantly higher number of emojis compared to Person 1. Person 2 utilized a total of 3,767 emojis, while Person 1 used a mere 70. On average, Person 2 was almost 54 times more likely to include an emoji in their message than Person 1. Additionally, Person 2 employed 43,177 capital letters, whereas Person 1 used only 20,729. This indicates that Person 2 was twice as likely to capitalize a given character. Person 1 employed a total of 23,947 words not found in the Brown Corpus, whereas Person 2 used 33,200. However, Person 2 had an average of 0.19539 unknown words per message, slightly lower than Person 1's average of 0.20312, despite using fewer unknown words overall. This discrepancy can be

attributed to Person 2 using nearly 50% more words per message (6.7782) compared to Person 1 (4.5578). The exploratory data analysis highlights that Person 1's messages are generally shorter with fewer unknown words, capital letters, and emojis, while Person 2's messages tend to be longer, incorporating more capital letters and emojis but also featuring more unknown words.

Graph 1. Exploratory Data Analysis



Graph 1 illustrates the relative ratio of features extracted from Person 1 and Person 2 to the overall mean of the dataset. For instance, it represents the average characters of Person 1 over the average characters of the entire dataset.

- Please note that spelling was calculated based on an English database. Consequently, any messages containing slang, typos, or in languages other than English are considered errors.

3. METHODOLOGY

3.1 Heuristics

The initial classification approach involved extracting trends identified in the exploratory data analysis as features for an SVM model. Among these heuristics, emoji frequency emerged as particularly informative. However, as not all messages contained emojis, additional factors such as average word length and capital letter frequency were included in the feature vector. Average character number in a message was omitted due to its redundancy with average words, and punctuation frequency was excluded because of the close values observed for Person 1 and Person 2.

This model was selected as the starting point to establish a non-machine learning baseline for comparing subsequent models. Leveraging heuristics allowed us to capture any outlier features inherent to the dataset. Furthermore, heuristics offered a dataset-specific approach to counteract potential cold start issues that might be encountered by other models.

Table 3.1.1. 5-Fold Cross Validation with Heuristic Features Only

	1	2	3	4	5	mean
Validation Accuracy	0.758	0.760	0.751	0.754	0.755	0.7556
Training Accuracy	0.755	0.755	0.757	0.756	0.754	0.7554

Interestingly, it's worth noting that this model exhibited a higher validation accuracy than training accuracy—a pattern that was not consistently observed throughout the rest of the study.

3.2 TF-IDF

3.2.1 Basics of TF-IDF

Term frequency-inverse document frequency, abbreviated as TF-IDF, serves as a metric indicating the likelihood of a specific word appearing in a given document, adjusted for the word's frequency across all documents in the corpus. In this task, TF-IDF offers insight into how probable it is for Person 1 or 2 to use a particular word. The calculation involves determining the term frequency of a word in a message and multiplying it by the logarithm of the inverse of the word's frequency across all messages.

$$tf(w, d) = \frac{f_{w,d}}{\sum_{w' \in d} f_{w',d}}$$

$$idf(w, D) = \log \frac{1 + N}{1 + |\{d \in D : w \in d\}|}$$

$$tfidf(w, d, D) = tf(w, d) \cdot idf(w, D)$$

where w is a given word, $f_{w,d}$ is the number of occurrences of word w in message d , and N is the total number of messages in the dataset D . We chose to add 1 to the denominator of IDF to prevent division by 0, and we add 1 to the numerator accordingly. We split the dataset into messages sent by Person 1 and messages sent by Person 2 to compute two sets of TF-IDFs for each message. This was to enable the model to see the message's score on Person 1's TF-IDF and Person 2's TF-IDF to more accurately determine to whom a given message belongs to.

3.2.2 TF-IDF fixed-length serialization

Every message in the dataset possesses a TF-IDF value for each word, presenting a challenge when incorporating TF-IDFs into each feature. Given that the feature is input into both an SVM and a logistic

regressor, it is imperative that each feature has the same length to facilitate meaningful comparisons and interactions between data points. Since each message comprises a distinct set of unique words, simply including a two-dimensional array of all TF-IDF values for each word would result in variable-length features across the dataset.

To address this, we sought to make TF-IDF analysis meaningful by performing an operation on the list of values to ensure a consistent length every time. Consequently, we opted to pass the maximum TF-IDF value of all the words in a message to the message feature, rather than including all individual TF-IDFs.

3.2.3 Improvements over solely using heuristical data

The heuristic data offers insights into the qualities of the message itself rather than in relation to other messages or the document as a whole. Consequently, while it can reveal strong evidence of different patterns and writing styles, it does not establish patterns between messages sent by the same person, treating each message as independent.

To address this, TF-IDFs are calculated, explicitly considering the context of all other messages in the dataset—specifically, all messages sent by either Person 1 or Person 2. This approach facilitates the model's ability to identify and predict different writing patterns for each person. However, the inclusion of heuristic data in each feature remains beneficial. Relying solely on TF-IDF may cause the model to overlook individual traits of the message. Incorporating heuristic data provides additional message-specific factors for training, thereby balancing the overall approach.

3.2.4 Performance

Including maximum TF-IDF values significantly improved the accuracy of the model, bringing the average to around 78%.

Table 3.2.1 . 5-Fold Cross Validation with Heuristic Features and TF-IDF

	1	2	3	4	5	mean
Validation Accuracy	0.798	0.771	0.771	0.785	0.776	0.7802
Training Accuracy	0.871	0.873	0.870	0.871	0.872	0.8714

3.3 Word2Vec

3.3.1 Doc2Vec and Word2Vec

One of the most promising models we aimed to work with was Word2Vec. In contrast to TF-IDF, Word2Vec held the potential to capture syntax patterns that were previously overlooked in our models. During the implementation, we opted for a slight variation called Doc2Vec. Unlike Word2Vec, Doc2Vec considers an entire document instead of individual words, resulting in improved results for message analysis compared to averaging across Word2Vec representations of all words in the message.

Across all Doc2Vec implementations, the most influential hyperparameter was the vector size. This parameter not only impacted the message representation but also played a significant role in how the SVM regularized when attempting to ensemble Doc2Vec with other features. The representation by Doc2Vec increased the number of features for the SVM to train on, leading to potential issues of overfitting. This challenge was addressed through the use of a non-linear activation function, as described in section 3.3.2.

3.3.2 Doc2Vec and Non-Linearity

The initial and straightforward approach we adopted involved using the vector representation of the message as features for the SVM. The Doc2Vec model was trained on a combination of Person 1 and Person 2's training data. When combined with heuristics, this model achieved an accuracy rate of approximately 80%.

Table 3.3.1 . 5-Fold Cross Validation with Heuristic Features and Doc2Vec

	1	2	3	4	5	mean
Validation Accuracy	0.802	0.807	0.799	0.790	0.795	0.7986
Training Accuracy	0.950	0.948	0.949	0.950	0.950	0.9494

In the subsequent step of utilizing Doc2Vec, we introduced the application of an activation function. The objective was to prevent the SVM from treating each component of the Doc2Vec as an independent feature but rather to consider it as a single feature representing the entire Doc2Vec. This was achieved by initially passing the Doc2Vec through a logistic regressor and subsequently transmitting the result of this logistic regression to the SVM.

Table 3.3.2 . 5-Fold Cross Validation with Heuristic Features and Activated Doc2Vec

	1	2	3	4	5	mean
Validation Accuracy	0.805	0.811	0.813	0.817	0.813	0.812
Training Accuracy	0.888	0.882	0.884	0.887	0.886	0.886

3.3.3 Doc2Vec Difference

An alternative approach with Doc2Vec involved training two separate models instead of using a single model on all data. One model was trained for each class—Person 1 and Person 2. The feature passed to the SVM is the difference between these two models. When a message is input, both Doc2Vec models generate their respective representations of the message. Each model then computes a similarity score between the message and all other training messages within its class. The message is subsequently classified based on which class had a training message more similar to it.

Table 3.3.3 . 5-Fold Cross Validation with Heuristic Features and Doc2Vec Difference

	1	2	3	4	5	mean
Validation Accuracy	0.815	0.807	0.808	0.811	0.808	0.810
Training Accuracy	0.884	0.883	0.883	0.883	0.886	0.884

The results from Doc2Vec Difference and Activated Doc2Vec were remarkably similar, showing a marginal difference of only 0.002. This similarity arises because both methods essentially perform the same task—taking the vector representation of the message and applying a nonlinear function before passing it to the SVM.

Despite the close results, Doc2Vec lags behind the Activated Doc2Vec method in a significant aspect: speed. While 5-fold cross-validation with Activated Doc2Vec took approximately 7 minutes to run, Doc2Vec consistently took more than double that time, ranging from 15 to 20 minutes. This discrepancy is attributed to the fact that creating the feature vector in most Doc2Vec models involves running through the entire training dataset to find the most similar message to the given one. This significantly inflates the runtime, making it impractical, especially when considering that Activated Doc2Vec outperformed it.

3.4 Ensemble

3.4.1 Ensembling Basics

The final attempt made to squeeze any more performance out of the models was ensembling them all together into one. This meant that the feature vector for the svm was made up of the:

Offset +

EmojiCount + WordCount + CapitalLetterCount + SpellingErrorRate +

TFIDF Class 1 + TFIDF Class 2 +

Activated Doc2Vec + Difference Doc2Vec

The advantage of this model lay in its potential to capture features of the dataset that other models might overlook. By training them together, it aimed to fill in gaps that could have posed cold-start issues for other models. However, this benefit came at the expense of training time, as the cost to train the ensembled model was significantly higher, making hyperparameter tuning challenging. Attempts were made to tune hyperparameters without including the difference Word2Vec, as it is the most resource-intensive among all the models. Nevertheless, this remained an exceptionally time-consuming effort. Additionally, reintroducing the difference Word2Vec into the ensemble always necessitated additional tuning.

3.4.2 Ensembling Performance

Overall, the performance of the ensembled model was consistently better than any of the individual methods or pairings.

Table 3.4.1. 5-Fold Cross Validation with Ensembled Model

	1	2	3	4	5	mean
Validation Accuracy	0.821	0.823	0.822	0.816	0.811	0.819
Training Accuracy	0.875	0.883	0.878	0.883	0.887	0.881

This provided an increase in performance of around 0.7% compared to the highest accuracy of other models and a 6% increase in accuracy from heuristics only.

This increase in performance is notable however it came at a large cost in terms of resources to train and predict.

4. PREVIOUS LITERATURE

Most approaches to this problem use deep learning models to some extent. For example, in [3], gated recurrent unit and long short-term memory models

were employed for an authorship identification task, attributing authors on the Reuters_50_50 dataset (of 2,500 articles) with 69.1% accuracy. Our ensemble model was able to get a significantly higher accuracy of 81.7%, which suggests that an ensemble approach might be preferable over a deep learning approach to authorship attribution. [3] also achieved a 99.8% accuracy using a Siamese neural network approach. While that is indeed impressive, Discord messages simply contain less text than entire news articles, which gives less data to determine a person's writing style from, which greatly limits how accurate a model like our own can be. The idea that an ensemble approach can provide better results than any one model is also supported by the research done in [1], where a transformer was ensembled with other models such as XGBoost, Random Forest, and a multilayer perceptron to classify 143,000 news articles, giving a high success rate of 95%. Despite analyzing significantly more data with many more possible authors than the Reuters_50_50 dataset, this ensemble approach was still able to get a very high success rate. Overall, it appears that while our ensemble approach is preferable for authorship attribution, it would benefit significantly from using deep learning models such as transformers, which have been shown to outperform word2vec in nearly every case as per [3].

5. CONCLUSIONS

In conclusion, we successfully developed a model capable of effectively performing the binary task of authorship analysis. Both TF-IDF and Doc2Vec strategies proved effective in enhancing the model's accuracy, with Doc2Vec being the more impactful of the two. This could be attributed to Doc2Vec's increased capability to extract meaning from the entire message, in contrast to TF-IDF, which examines words in a message individually.

Remarkably, the heuristics-only model demonstrated surprising accuracy, achieving a 75% success rate—far superior to random guessing. However, a notable drawback was the scalability of the model. Heuristics were tailored to the specific dataset, requiring reevaluation for each new problem. Moreover, TF-IDF and difference Doc2Vec calculations needed to be trained for each class, posing challenges for expanding the model to a multiclass setting. Despite similarities in performance, activated Doc2Vec is recommended over difference Doc2Vec.

Future research avenues may explore the use of different deep learning models and hyperparameter tuning. Additionally, alternative models to SVM (utilized to combine features) or logistic regression (used as the activation function of Doc2Vec) could be investigated. Further, the adoption of transformers, instead of Doc2Vec, has shown potential for outperforming the models used in our research. While there remains much to explore in the field of authorship attribution, our findings suggest that ensemble models are the optimal approach to the problem, as opposed to relying on a single model.

REFERENCES

- [1] Ahmed Abbasi, Abdul Rehman Javed, Farkhund Iqbal, Zunera Jalil, Thippa Reddy Gadekallu, and Natalia Kryvinska. 2022. Authorship identification using ensemble learning. *Scientific Reports* 12, 1. <http://doi.org/10.1038/s41598-022-13690-4>
- [2] Julian von der Mosel, Alexander Trautsch, and Steffen Herbold. 2023. On the validity of pre-trained transformers for natural language processing in the Software Engineering Domain. *IEEE Transactions on Software Engineering* 49, 4: 1487–1507. <http://doi.org/10.1109/tse.2022.3178469>
- [3] Chen Qian, Tianchang He, and Rao Zhang. 2017. Deep Learning based authorship identification. 2760185. Retrieved December 2, 2023 from <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1174/reports/2760185.pdf>